

```
In [ ]: # Arithmetic operator
```

```
In [1]: x1 = 10  
        y1 = 5
```

```
In [2]: x1 + y1
```

```
Out[2]: 15
```

```
In [3]: x1 - y1
```

```
Out[3]: 5
```

```
In [4]: x1 * y1
```

```
Out[4]: 50
```

```
In [5]: x1 / y1
```

```
Out[5]: 2.0
```

```
In [6]: x1 // y1
```

```
Out[6]: 2
```

```
In [7]: x1 ** y1
```

```
Out[7]: 100000
```

assignment operator

```
In [5]: x = 2
```

```
In [6]: x = x + 2  
        x
```

```
Out[6]: 4
```

```
In [7]: 4 + 10
```

```
Out[7]: 14
```

```
In [8]: _ + 4
```

```
Out[8]: 18
```

```
In [9]: x += 2  
        x
```

```
Out[9]: 6
```

```
In [11]: x *=2  
x
```

```
Out[11]: 24
```

```
In [13]: x -=2  
x
```

```
Out[13]: 20
```

```
In [15]: x /=2  
x
```

```
Out[15]: 5.0
```

```
In [17]: x //=2  
x
```

```
Out[17]: 1.0
```

Unary operator

```
In [18]: n = 7  
n
```

```
Out[18]: 7
```

```
In [20]: m = -n  
m
```

```
Out[20]: -7
```

```
In [21]: n
```

```
Out[21]: 7
```

```
In [22]: # Relational Operator
```

```
In [23]: a = 5  
b = 6
```

```
In [24]: a < b
```

```
Out[24]: True
```

```
In [25]: a > b
```

```
Out[25]: False
```

```
In [26]: a == b
```

```
Out[26]: False
```

```
In [27]: a!=b
```

```
Out[27]: True
```

```
In [28]: b = 5
```

```
In [29]: a == b
```

```
Out[29]: True
```

Logical operator

```
In [31]: a = 5  
b = 4
```

```
In [32]: a < 8 and b < 5
```

```
Out[32]: True
```

```
In [33]: a < 8 or b < 2
```

```
Out[33]: True
```

```
In [34]: a>8 or b<2
```

```
Out[34]: False
```

```
In [35]: x = False
```

```
In [37]: y = not(x)  
y
```

```
Out[37]: True
```

Number system conversion

```
In [39]: 25
```

```
Out[39]: 25
```

```
In [40]: bin(25)
```

```
Out[40]: '0b11001'
```

```
In [41]: int(0b11001)
```

```
Out[41]: 25
```

```
In [42]: bin(30)
```

```
Out[42]: '0b11110'
```

```
In [43]: int(0b11110)
```

```
Out[43]: 30
```

```
In [44]: oct(25)
```

```
Out[44]: '0o31'
```

```
In [45]: int(0o31)
```

```
Out[45]: 25
```

```
In [46]: int(0b11110)
```

```
Out[46]: 30
```

```
In [47]: hex(25)
```

```
Out[47]: '0x19'
```

```
In [48]: int(0x19)
```

```
Out[48]: 25
```

```
In [49]: hex(1)
```

```
Out[49]: '0x1'
```

```
In [50]: hex(10)
```

```
Out[50]: '0xa'
```

bitwise operator

```
In [51]: # complement ~ it gives next no with negative sign
```

```
~12
```

```
Out[51]: -13
```

```
In [52]: ~ 54
```

```
Out[52]: -55
```

```
In [53]: # And operator
```

```
In [54]: 12 & 13
```

```
Out[54]: 12
```

```
In [56]: 14 & 15
```

```
Out[56]: 14
```

```
In [57]: # or operator
```

```
In [58]: 12|13
```

```
Out[58]: 13
```

```
In [59]: 35 ^ 40
```

```
Out[59]: 11
```

```
In [60]: 40 ^ 70
```

```
Out[60]: 110
```

```
In [63]: 10<<1
```

```
Out[63]: 20
```

```
In [64]: 10<<2
```

```
Out[64]: 40
```

```
In [65]: 14>>1
```

```
Out[65]: 7
```

```
In [66]: 14>>2
```

```
Out[66]: 3
```

```
In [ ]:
```